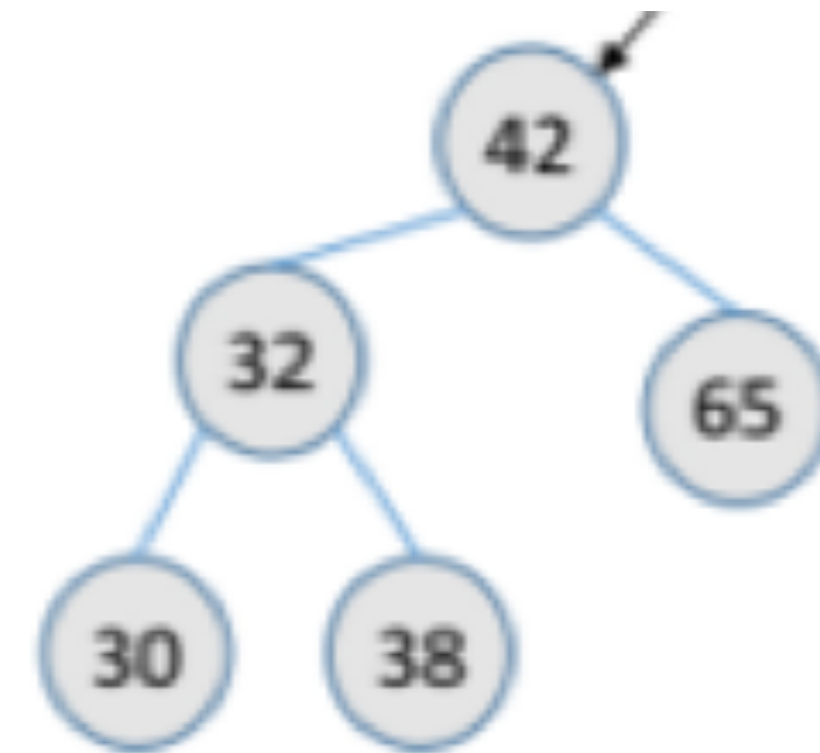


Lecture 23

Trees and heaps

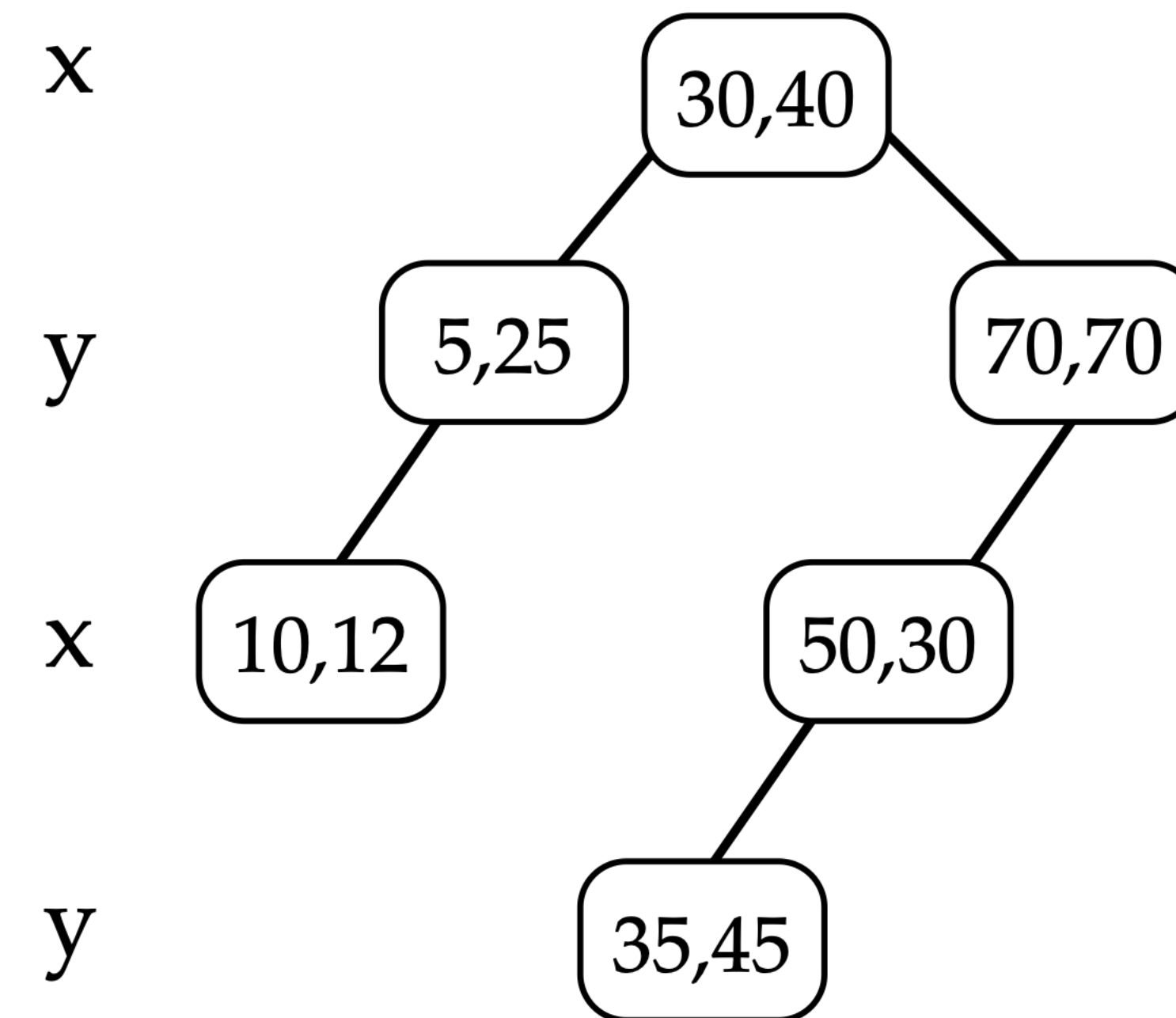
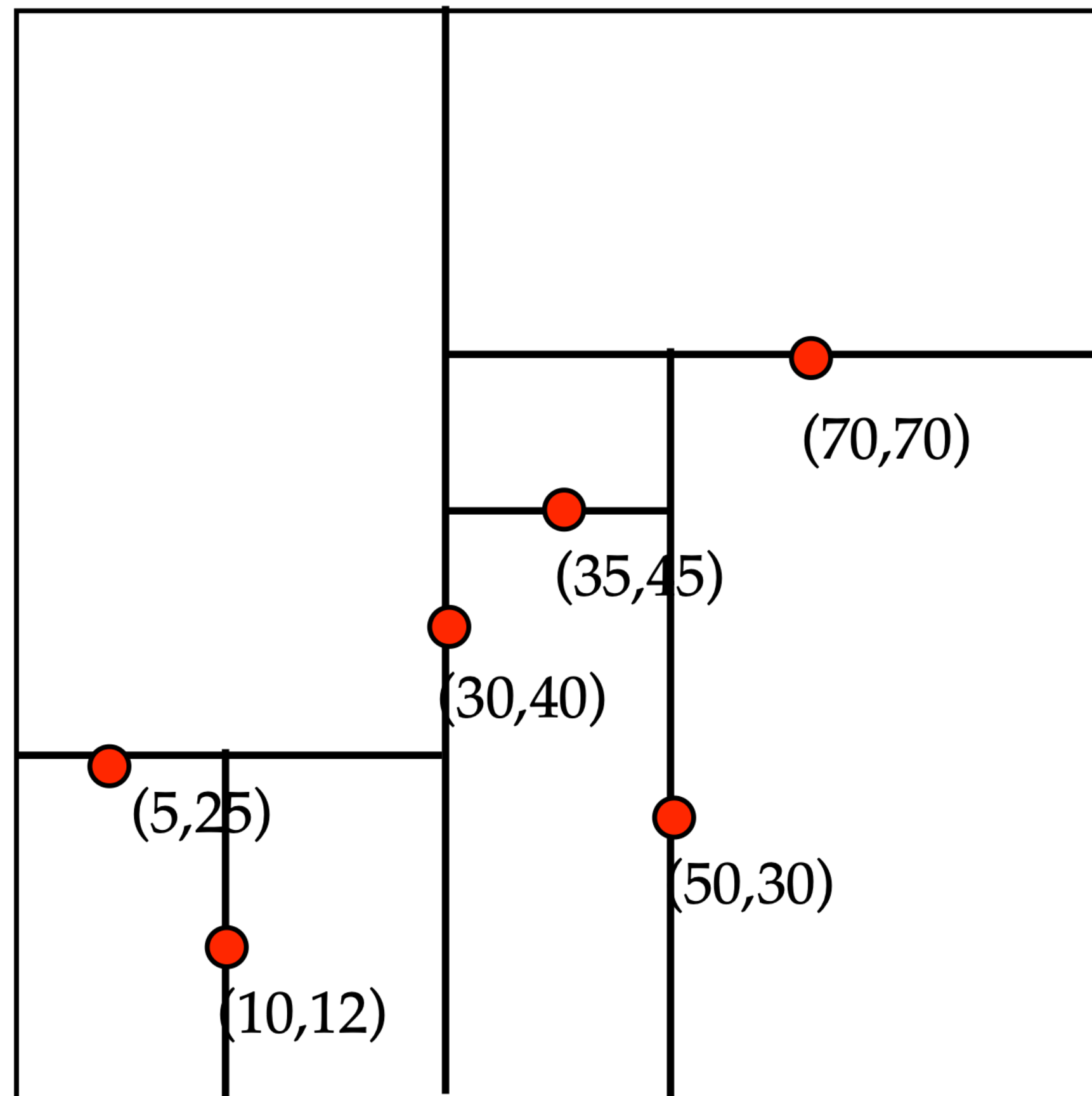
Removing elements in BSTs

- There are three cases
 - The removing node is a leaf-node
 - The removing node has 1 child (left/right)
 - The removing node has 2 children



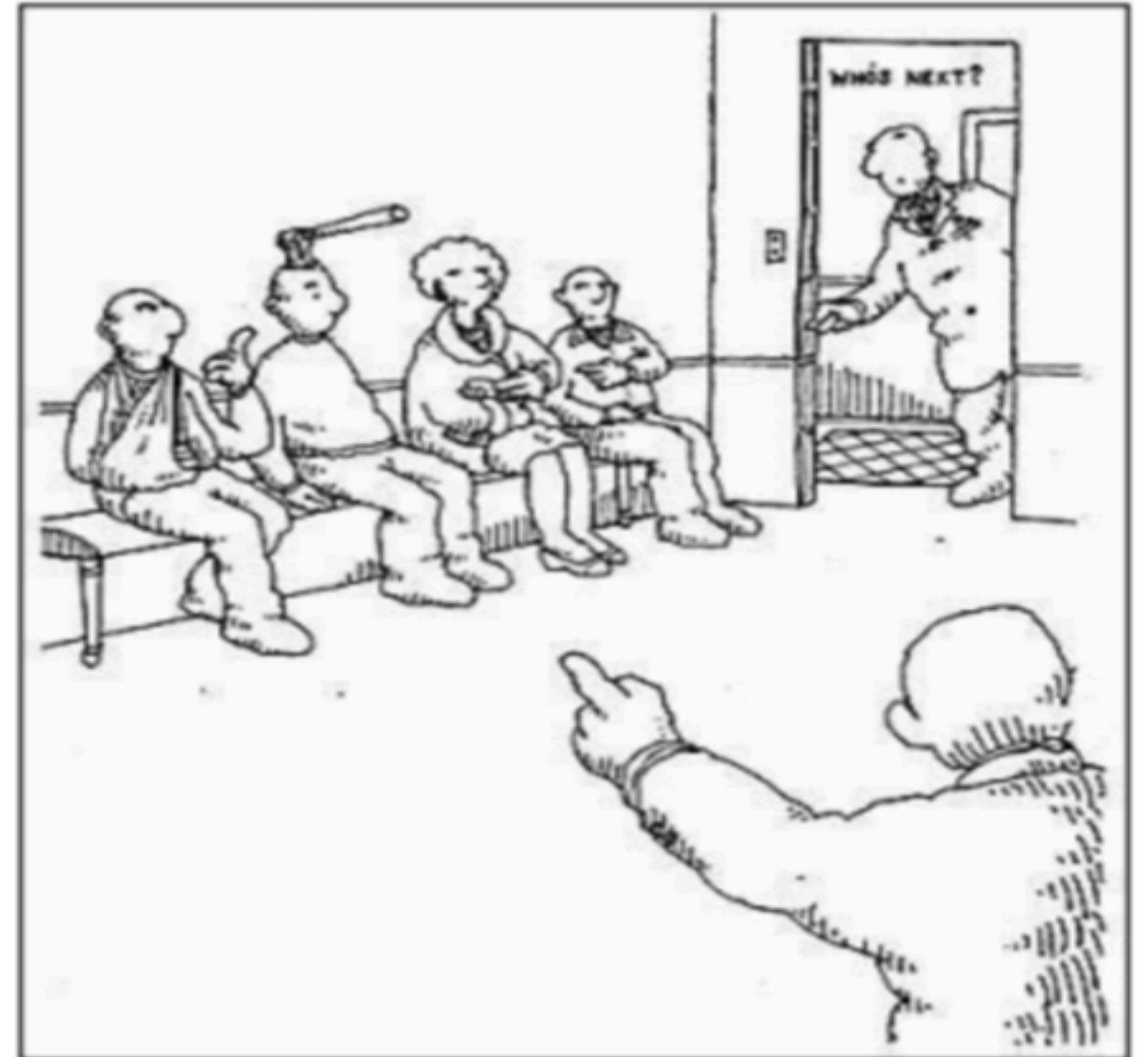
kd-Trees - BSTs in multiple dimensions

insert: (30,40), (5,25), (10,12), (70,70), (50,30), (35,45)



Priority Queue (aka Heap) ADT

- Emergency Room of a hospital operates as a priority queue
 - Patients sorted according to severity, NOT how long they have waited
- Used in any situation when you want to be able to access the maximum (or minimum) element very quickly.

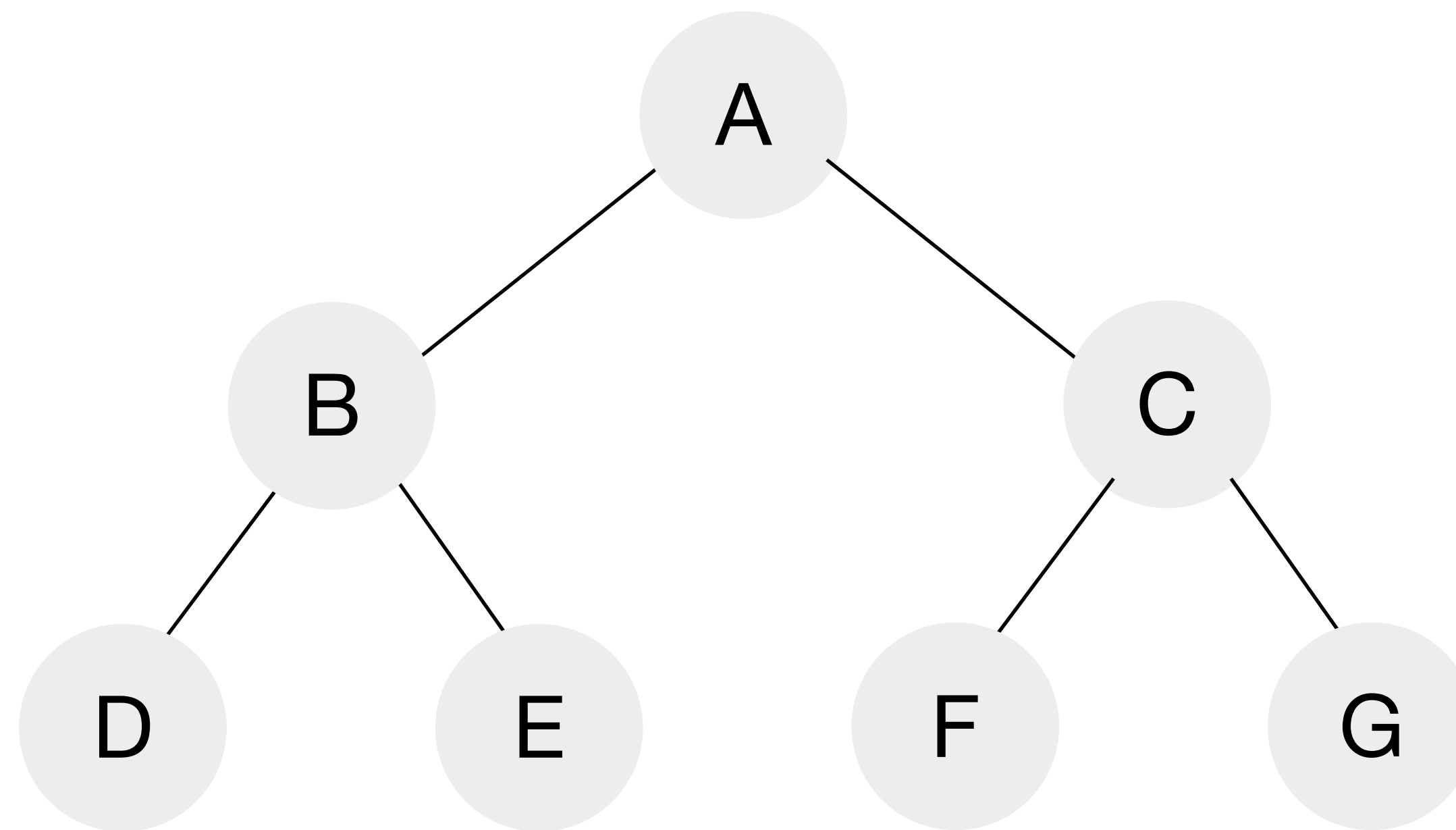


“sorted” linked list is not fast enough

- Removal will be fast, $O(1)$
- But insertion takes $O(n)$

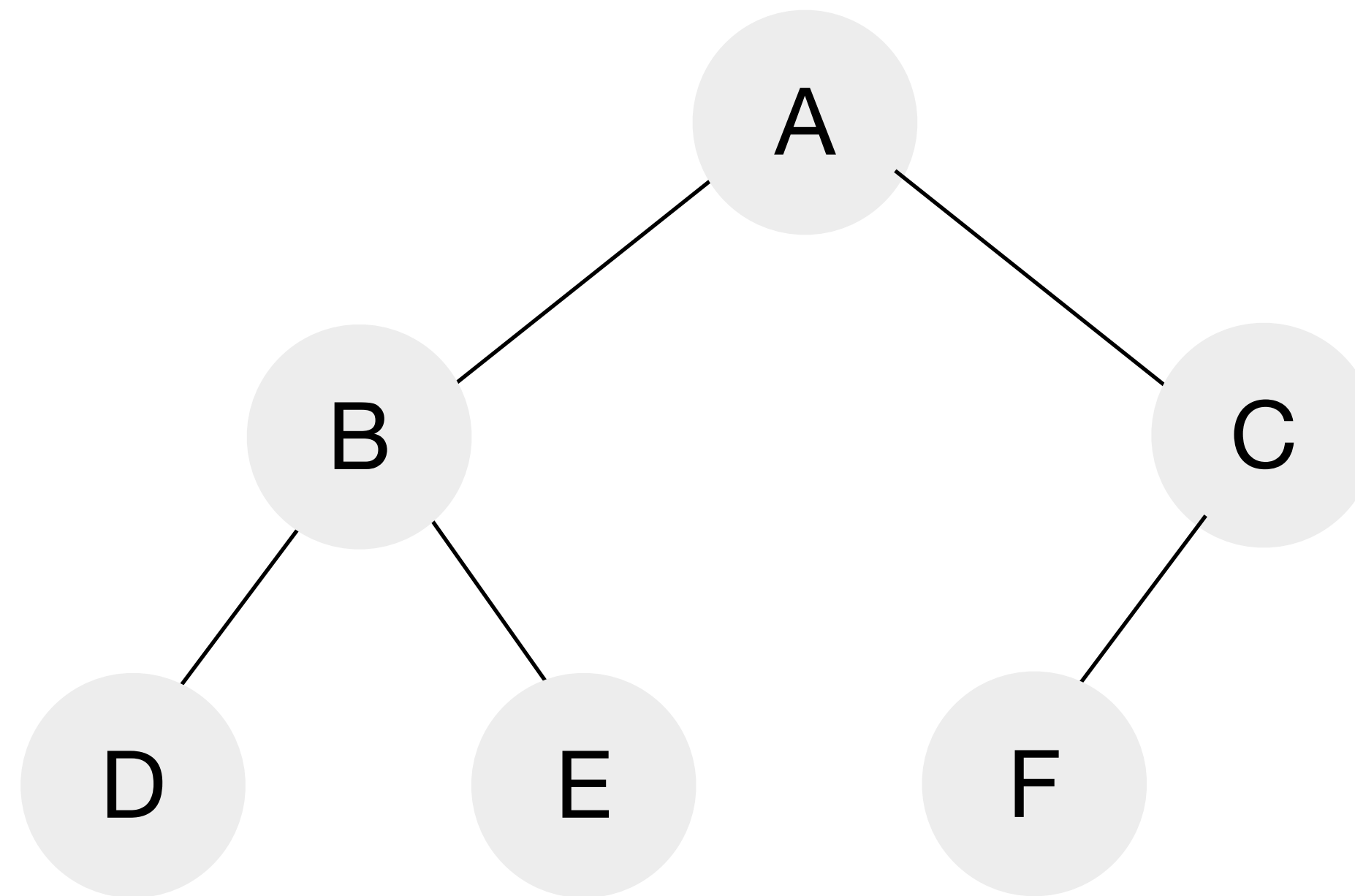
Heaps builds on complete binary trees

- In a complete binary tree, every level is completely filled and all nodes in the last level are as far left as possible (unless the node is at the last level)



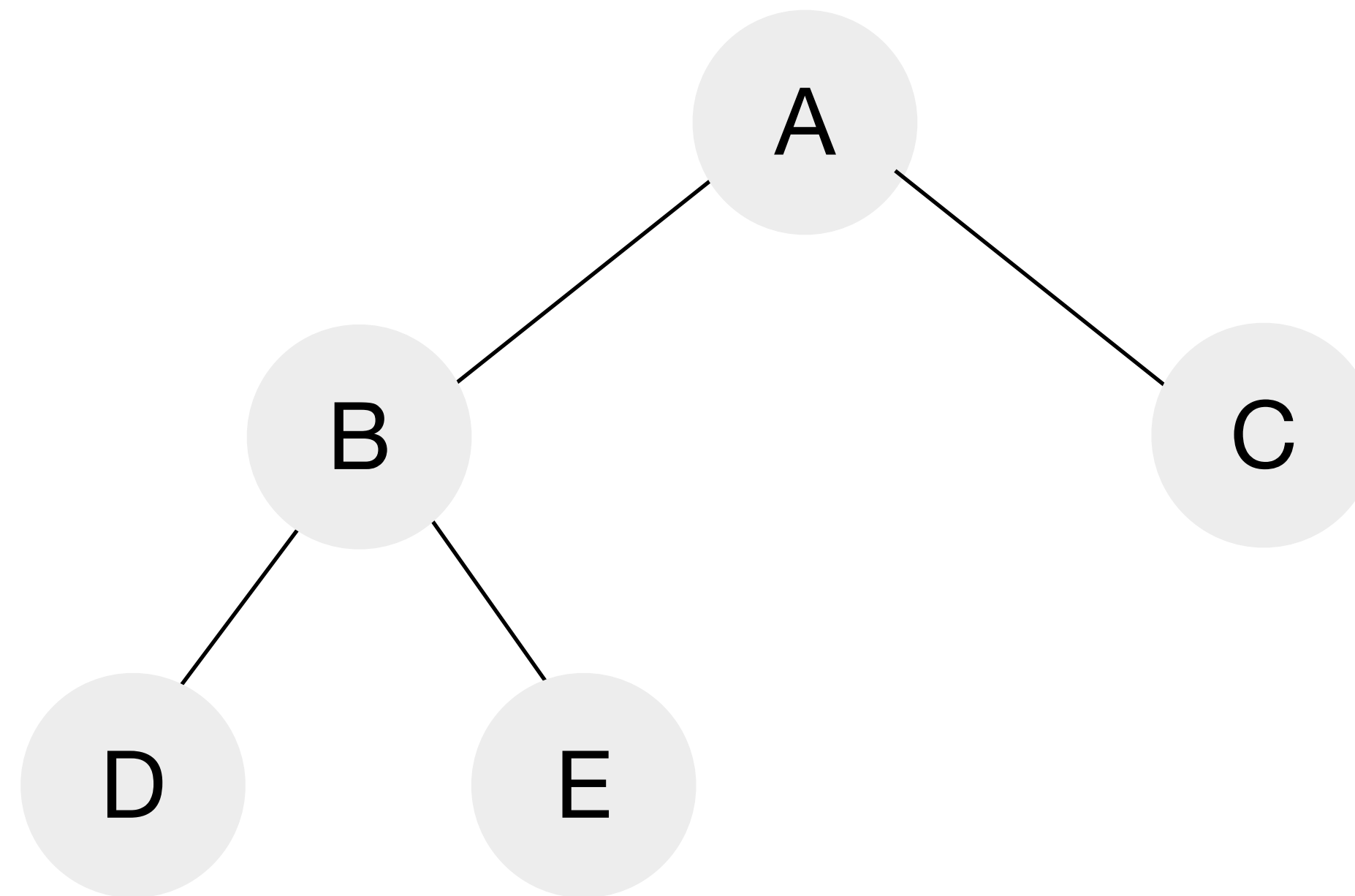
Heaps builds on complete binary trees

- In a complete binary tree, every level is completely filled and all nodes in the last level are as far left as possible (unless the node is at the last level)



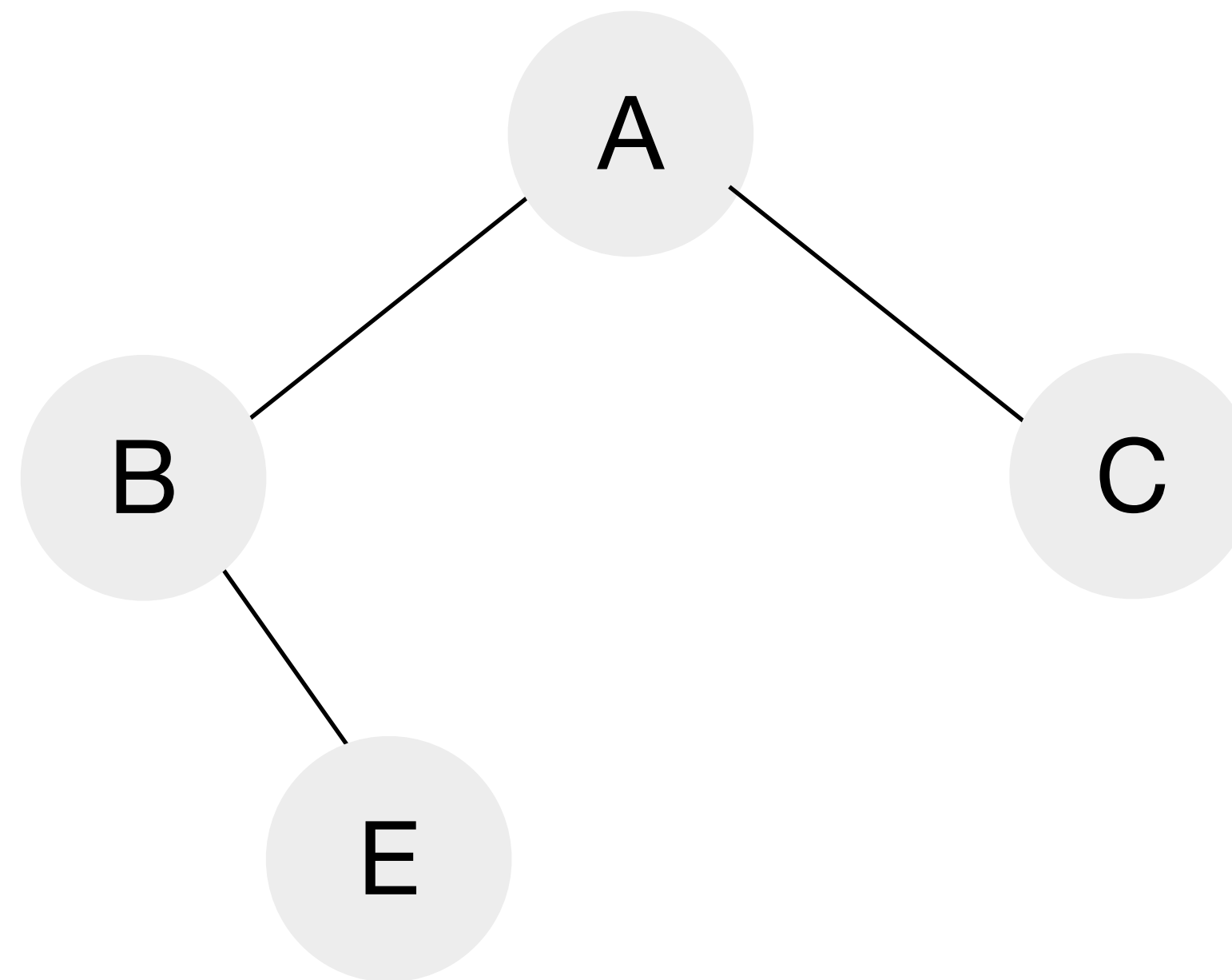
Heaps builds on complete binary trees

- In a complete binary tree, every level is completely filled and all nodes in the last level are as far left as possible (unless the node is at the last level)



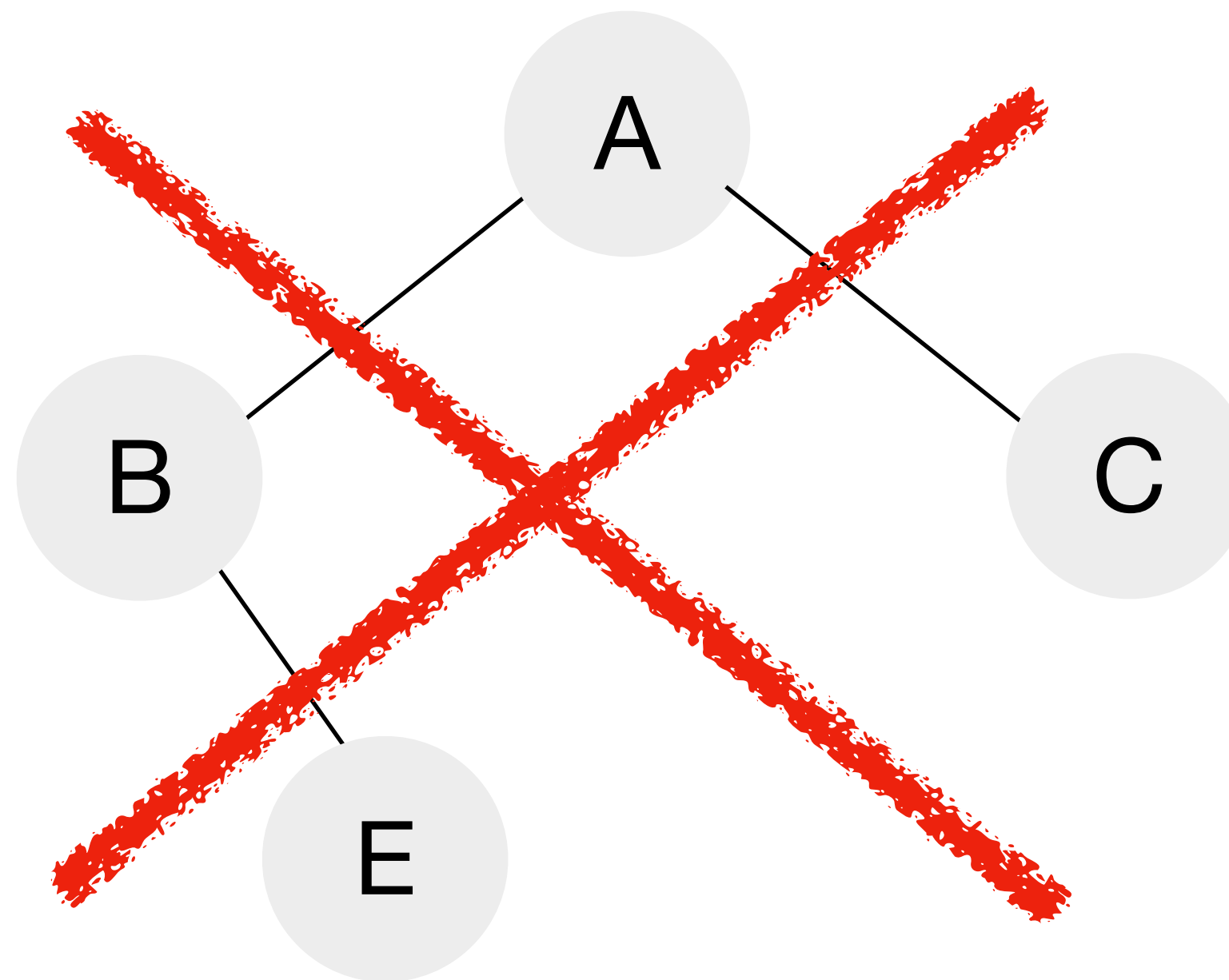
Heaps builds on complete binary trees

- In a complete binary tree, every level is completely filled and all nodes in the last level are as far left as possible (unless the node is at the last level)

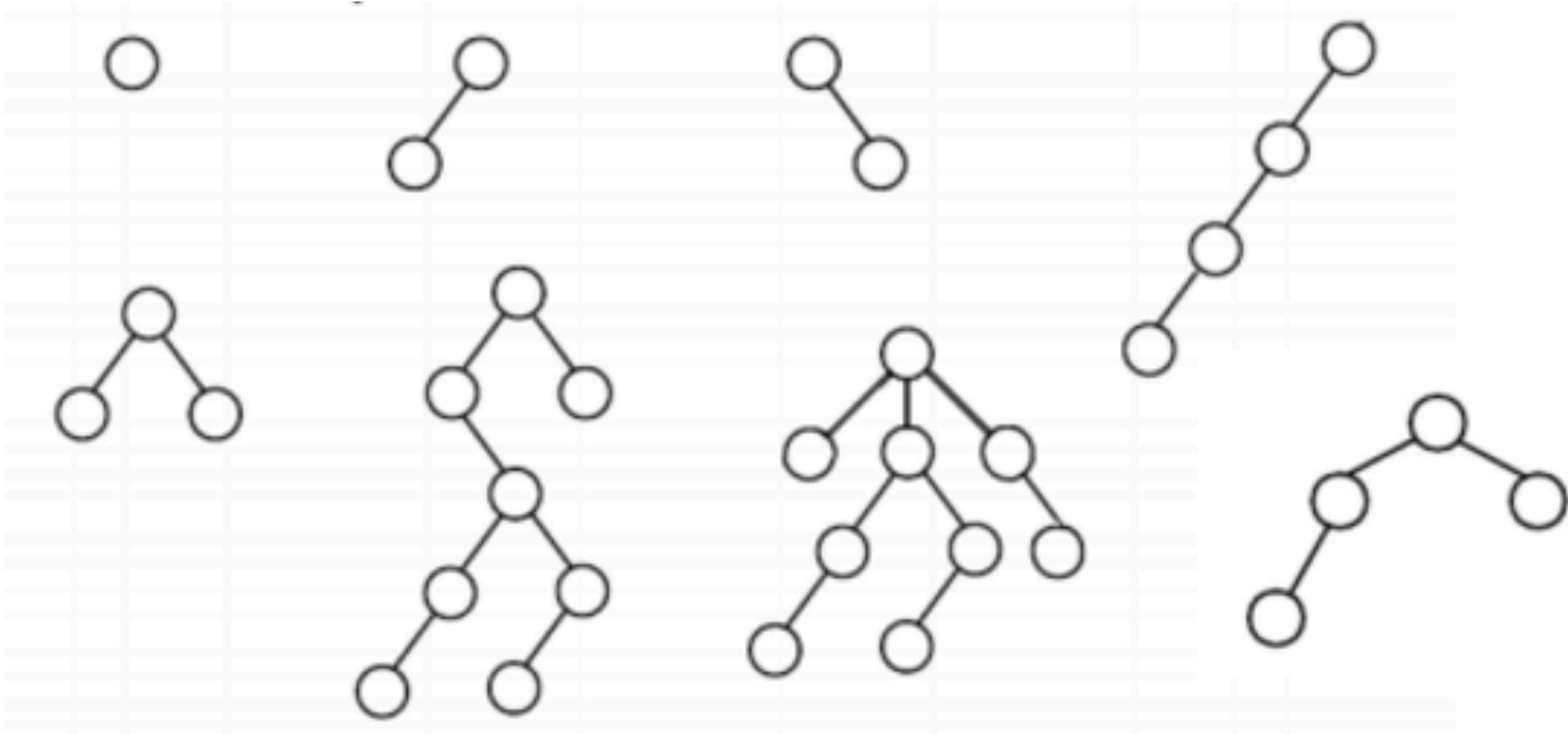


Heaps builds on complete binary trees

- In a complete binary tree, every level is completely filled and all nodes in the last level are as far left as possible (unless the node is at the last level)



How many of these could be valid heaps?



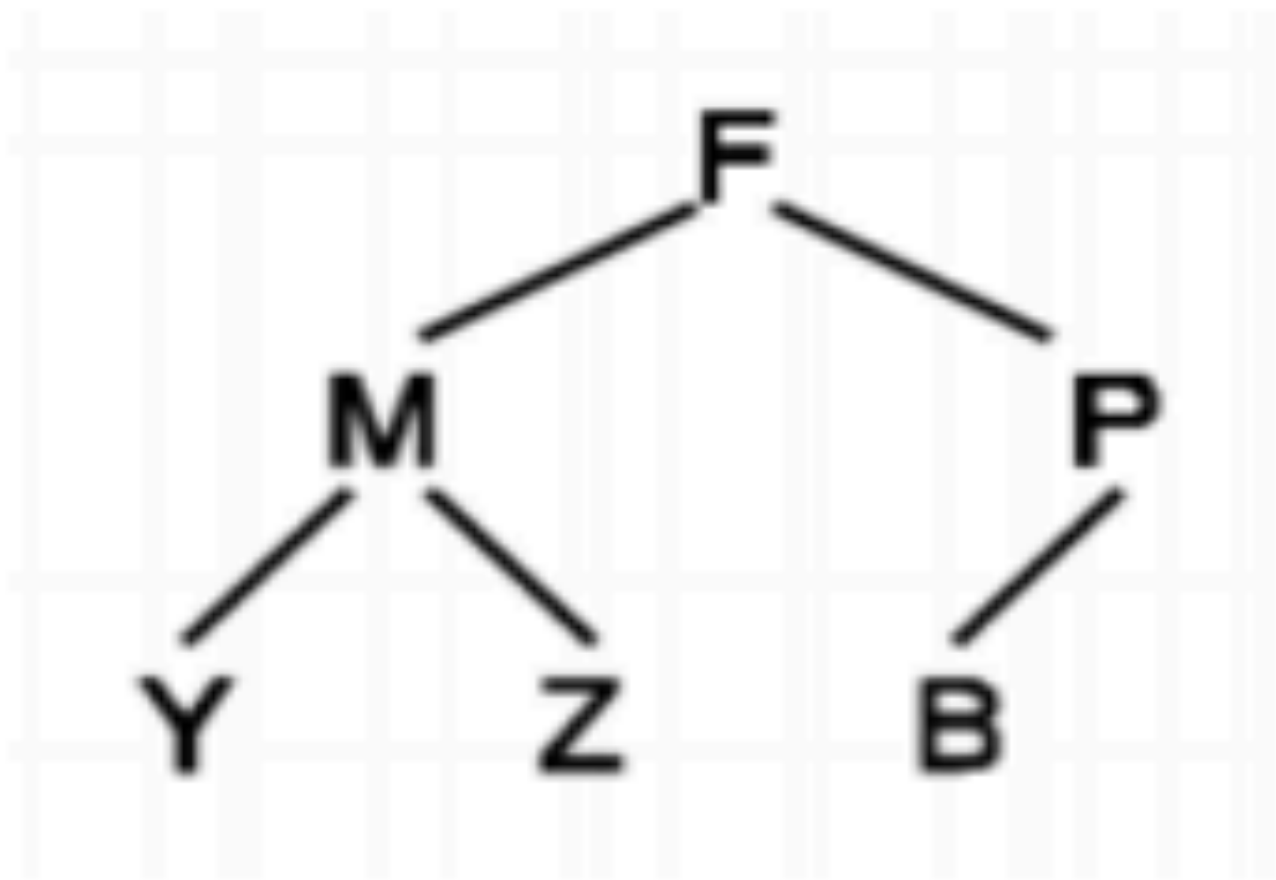
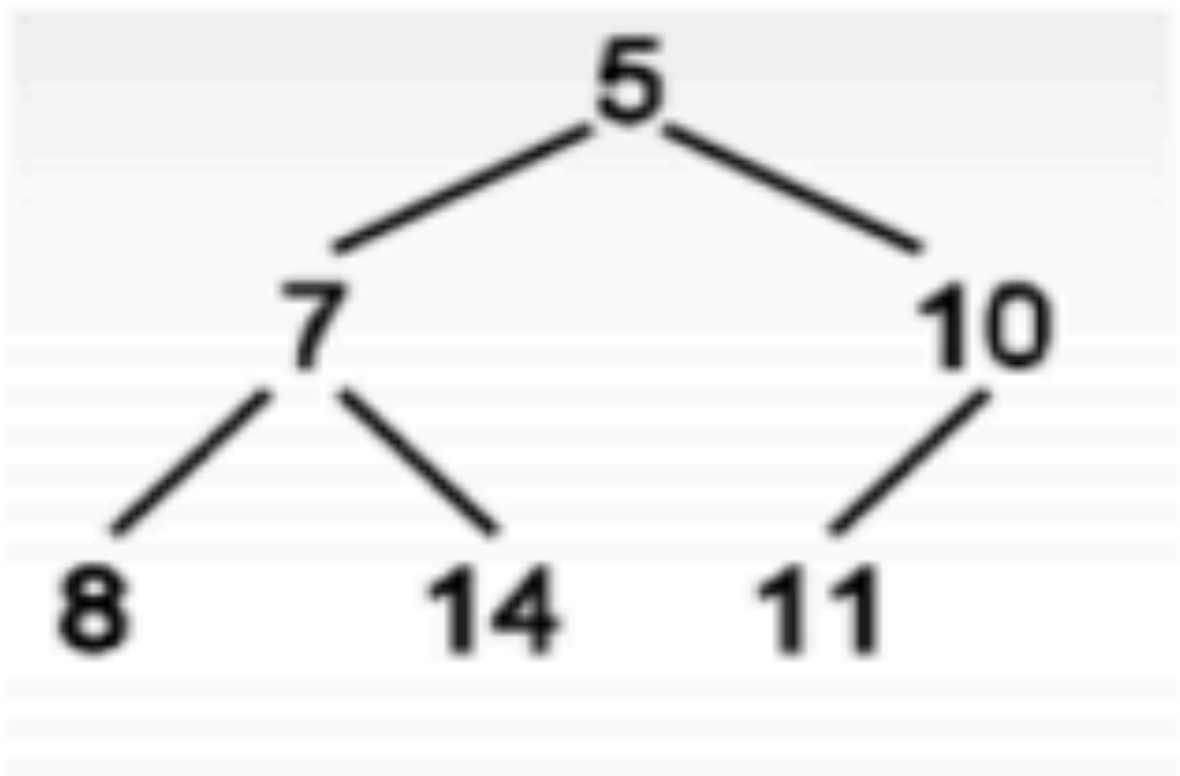
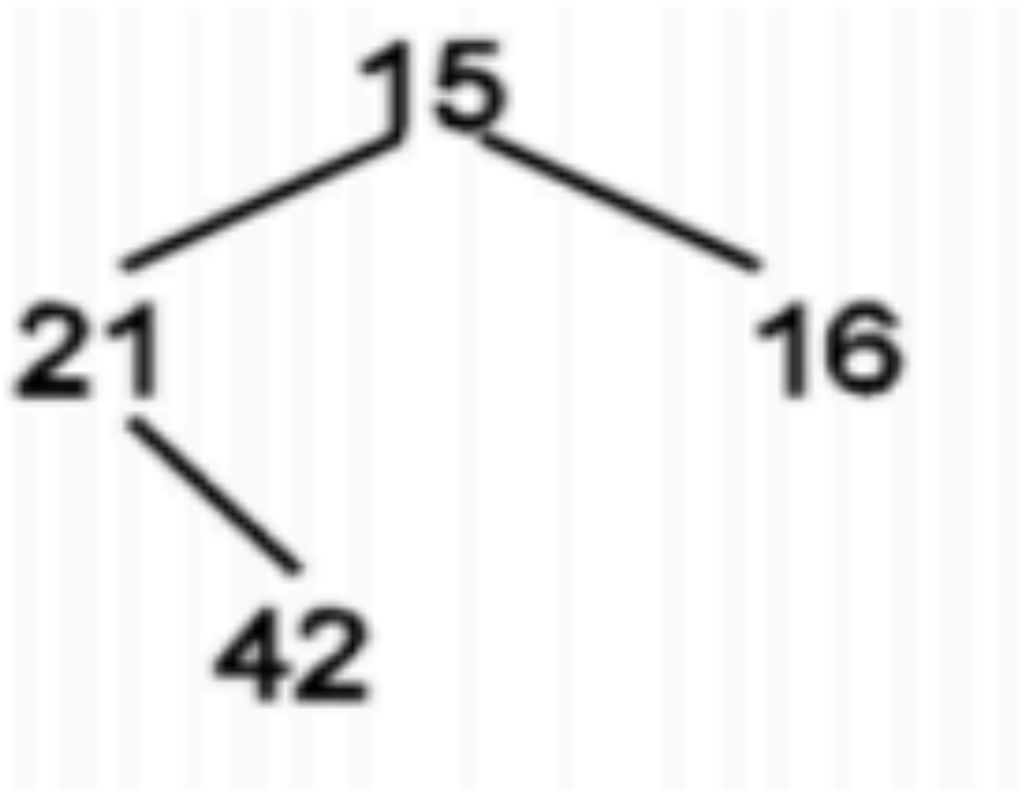
A	0-1
B	2
C	4
D	5-8

Class[Buzz]: valid_heaps

Heaps has some additional restrictions

- The tree (and recall it's a binary tree) must be complete
- Ordering of data must obey heap property
 - If it is Min-heap: a parent's data is always $\leq$ its children's data
 - If it is Max-heap: a parent's data is always $\geq$ its children's data

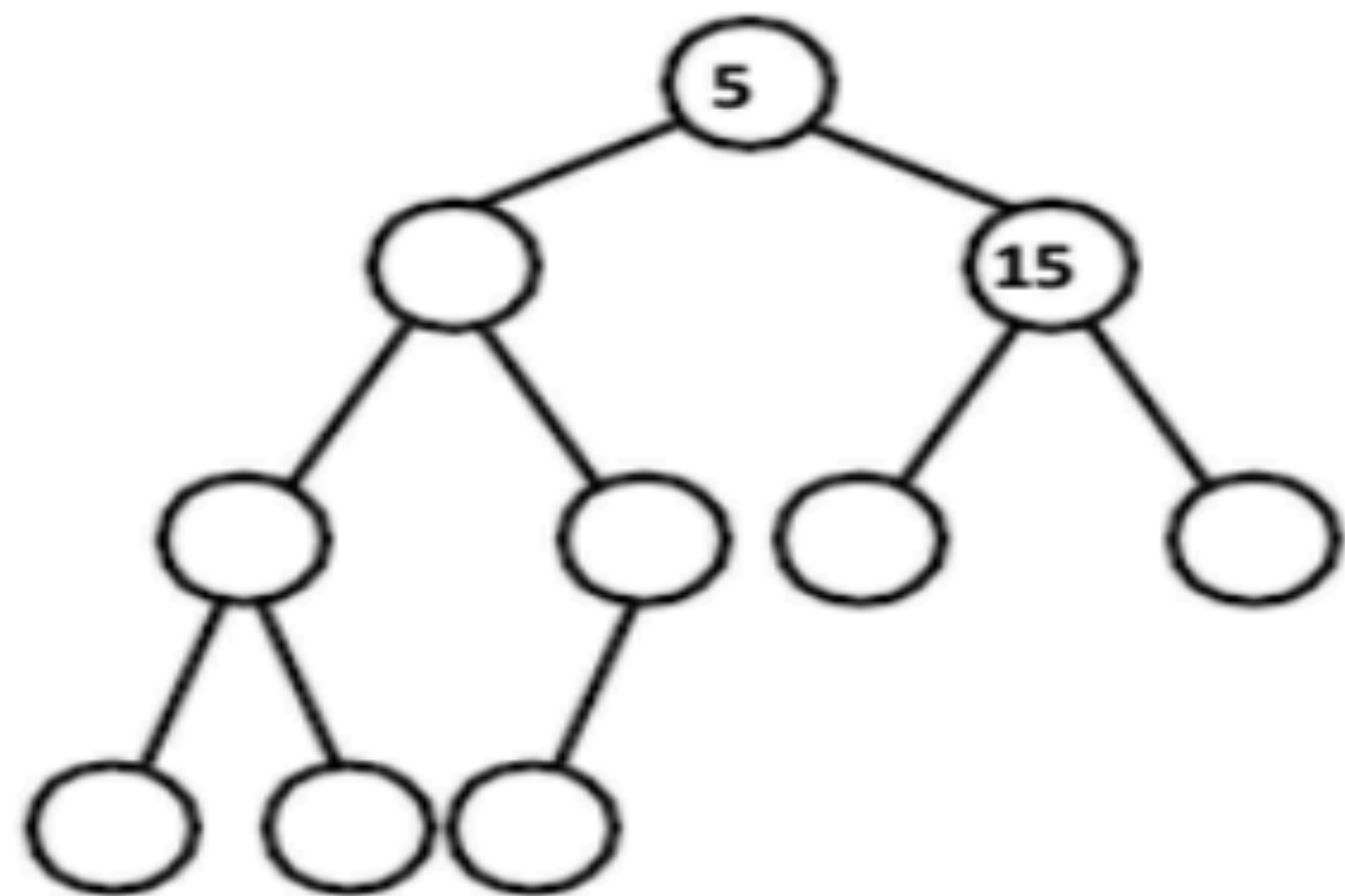
How many of these could be valid min-heaps?



A	0
B	1
C	2
D	3

Class[Buzz]: valid_min_heaps

In how many places could the number 35 be located in this min-heap?

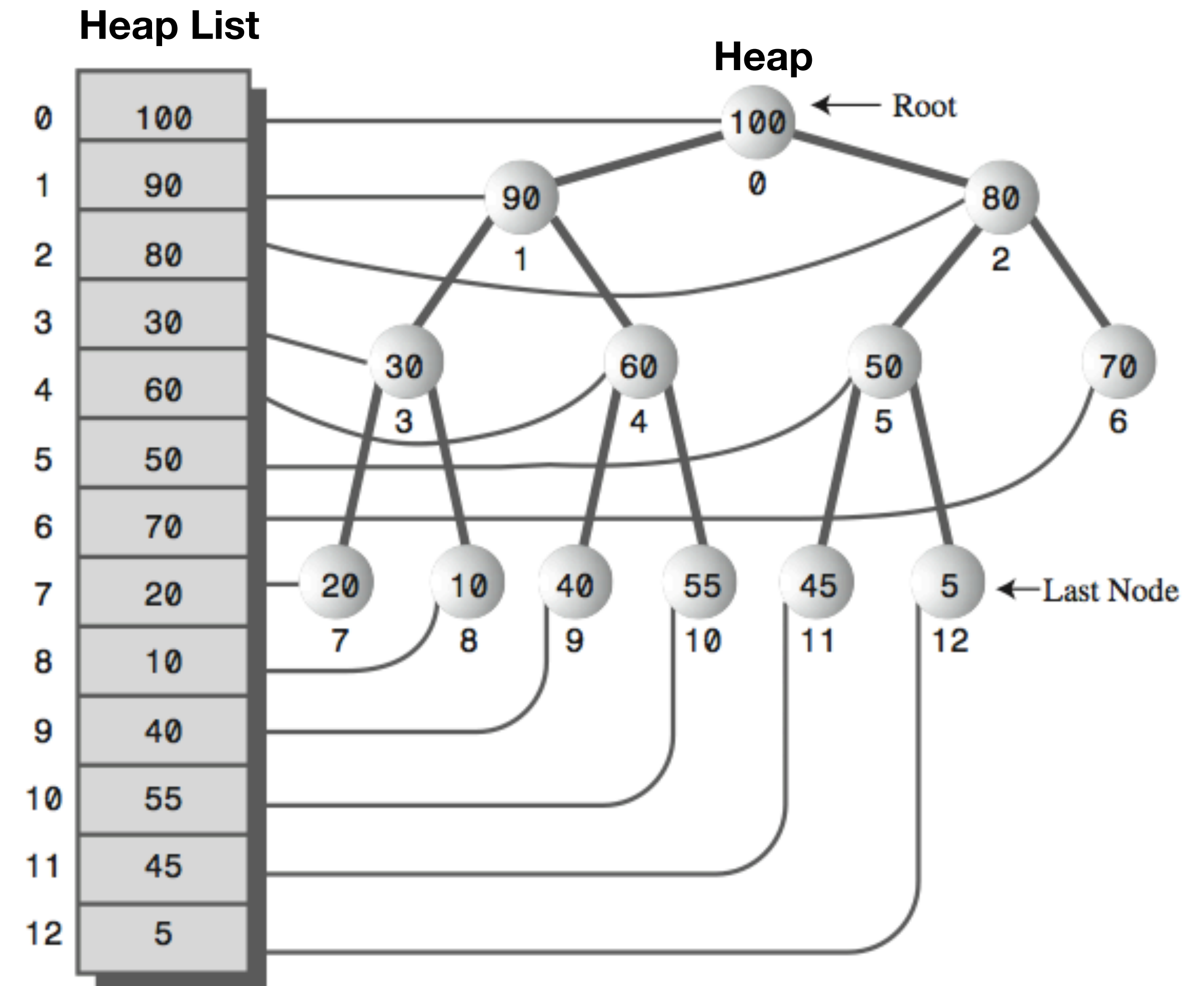


A	0-2
B	3-4
C	5-6
D	7-8

Class[Buzz]: min_heap_locations

Heaps are implemented simply with a list

- Because it's a complete binary tree

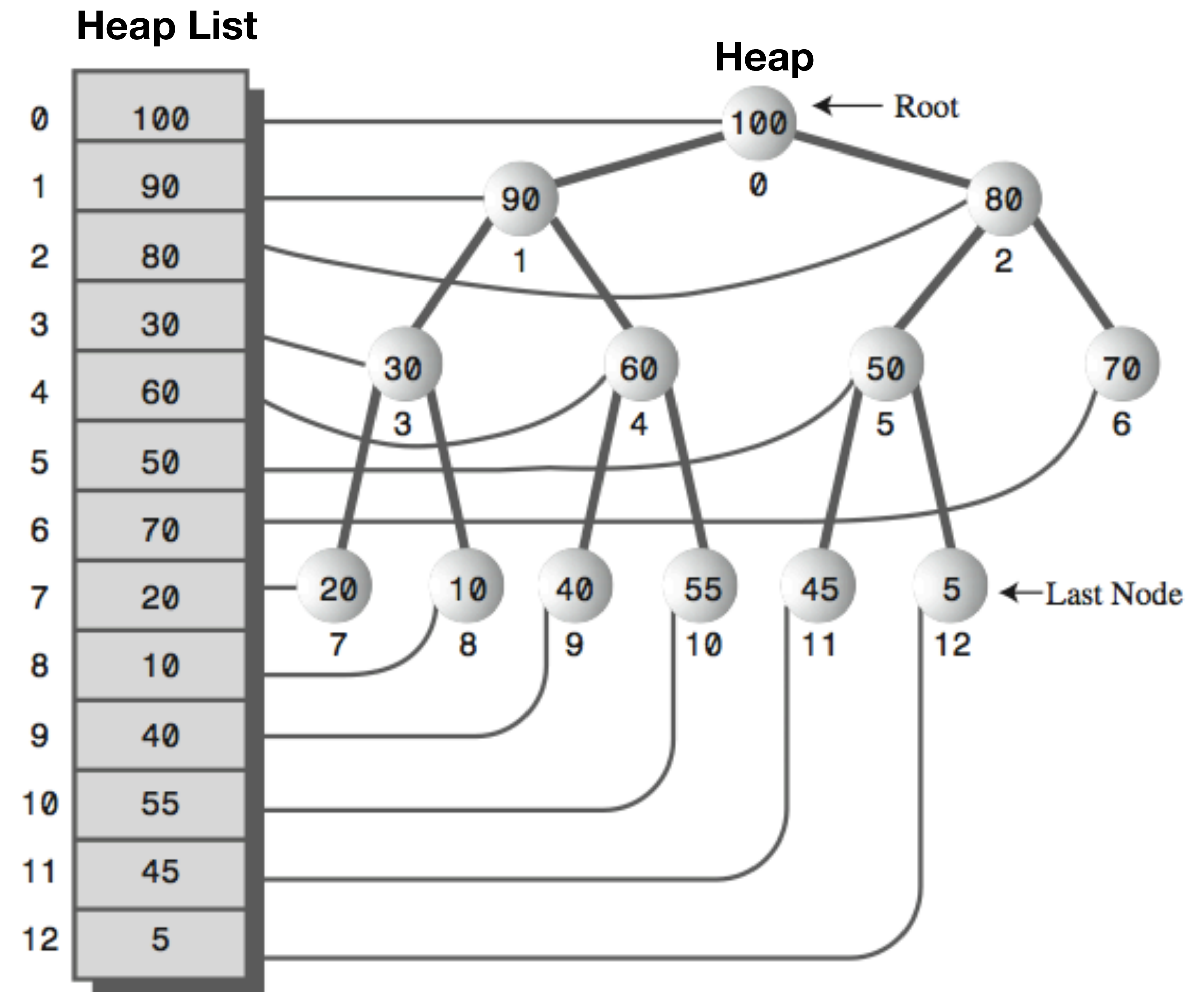


“Completeness” eases the access to heaps

For a node in array index i , what is the index of its parent?

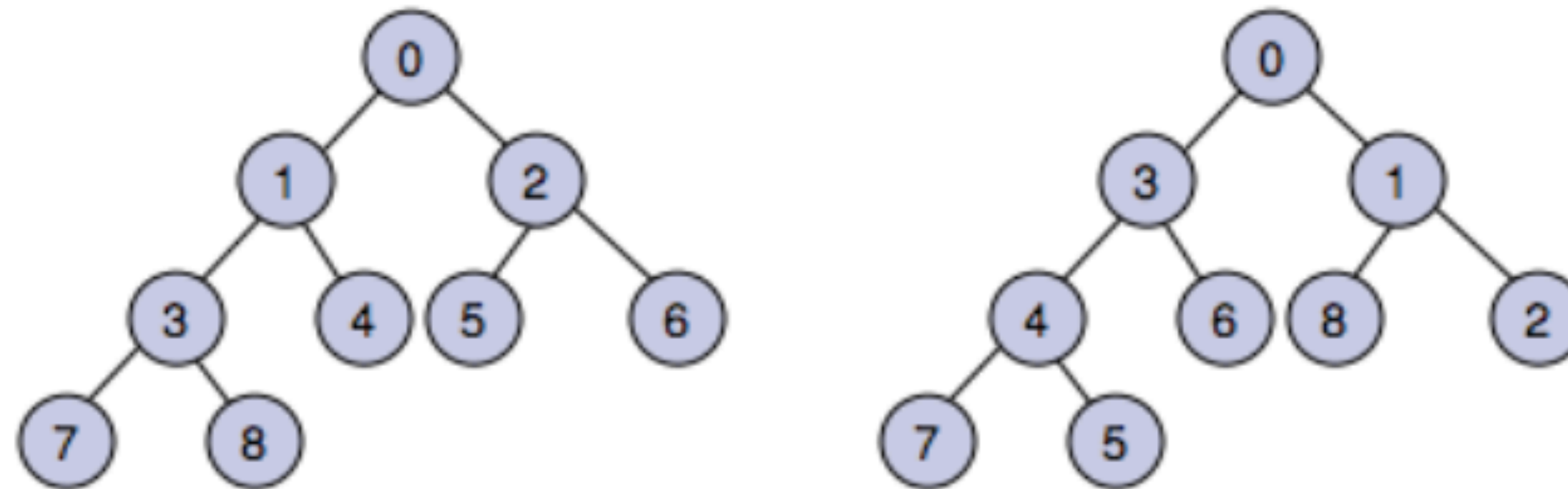
A	$i - 2$
B	$i / 2$
C	$(i - 1) / 2$
D	$2 * i$

Class[Buzz]: completeness



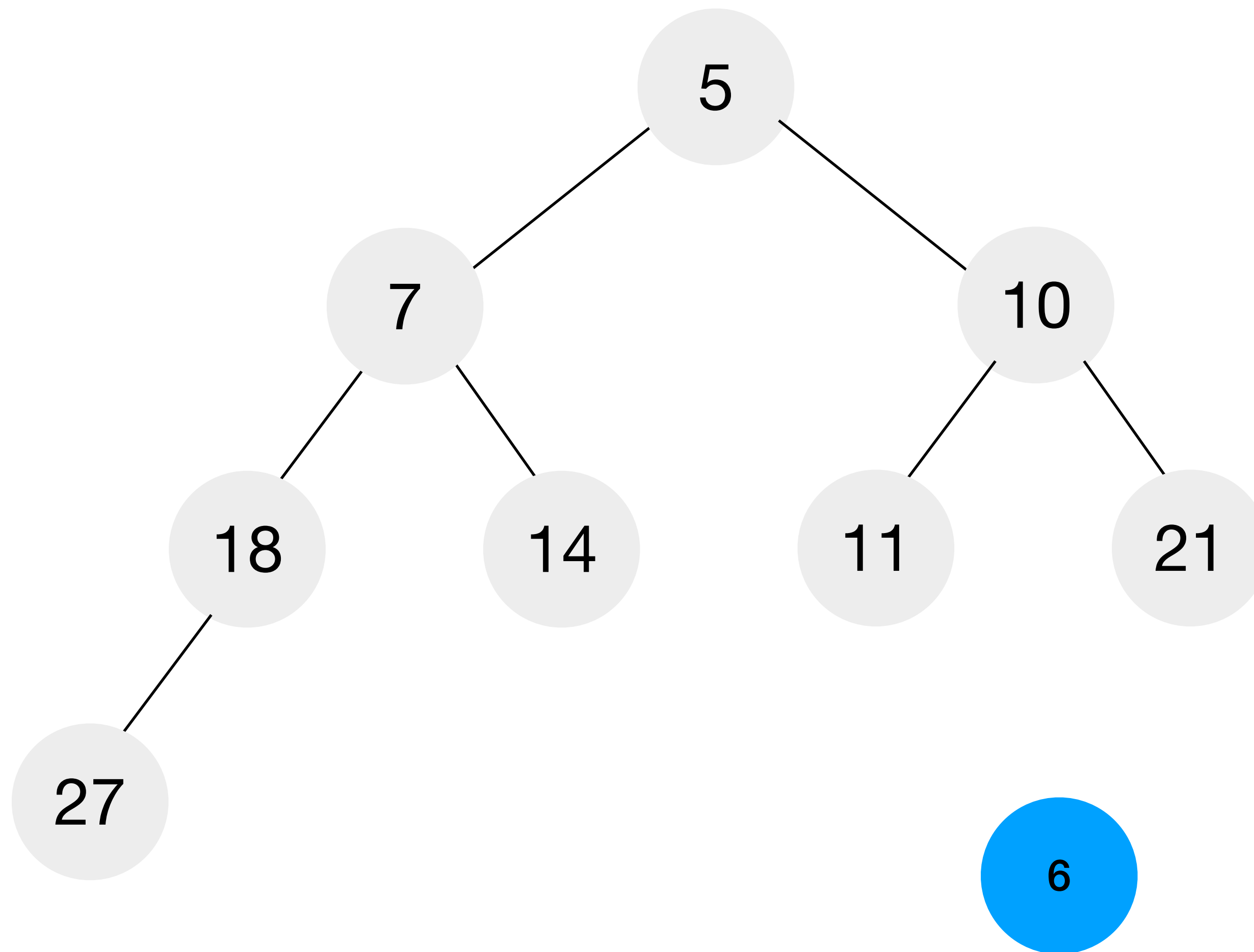
Multiple shapes are possible with heaps

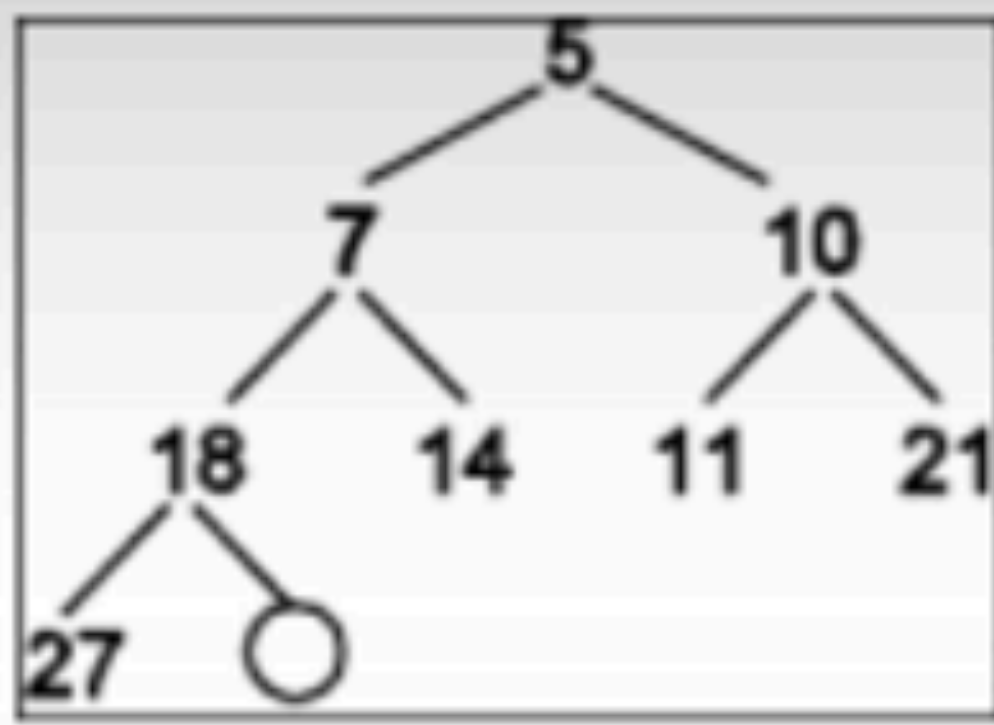
- Both trees below are all valid heaps on the numbers 0-8



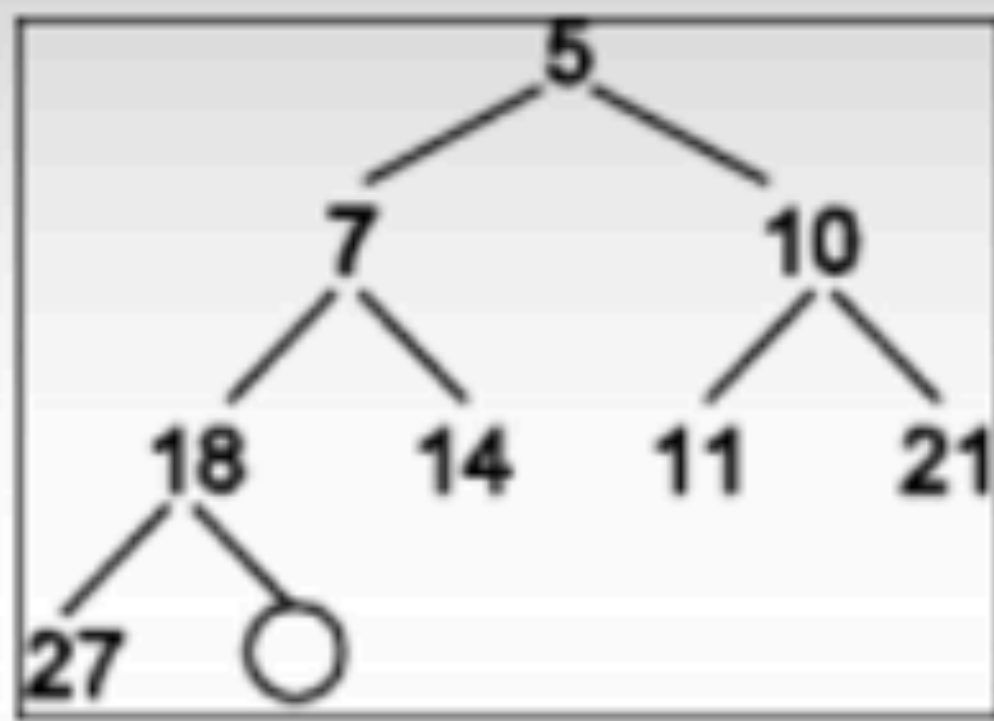
- Every heap with same number of nodes has the same shape (to fulfill the completeness property)

Insertion: insert at the end then bubble-up

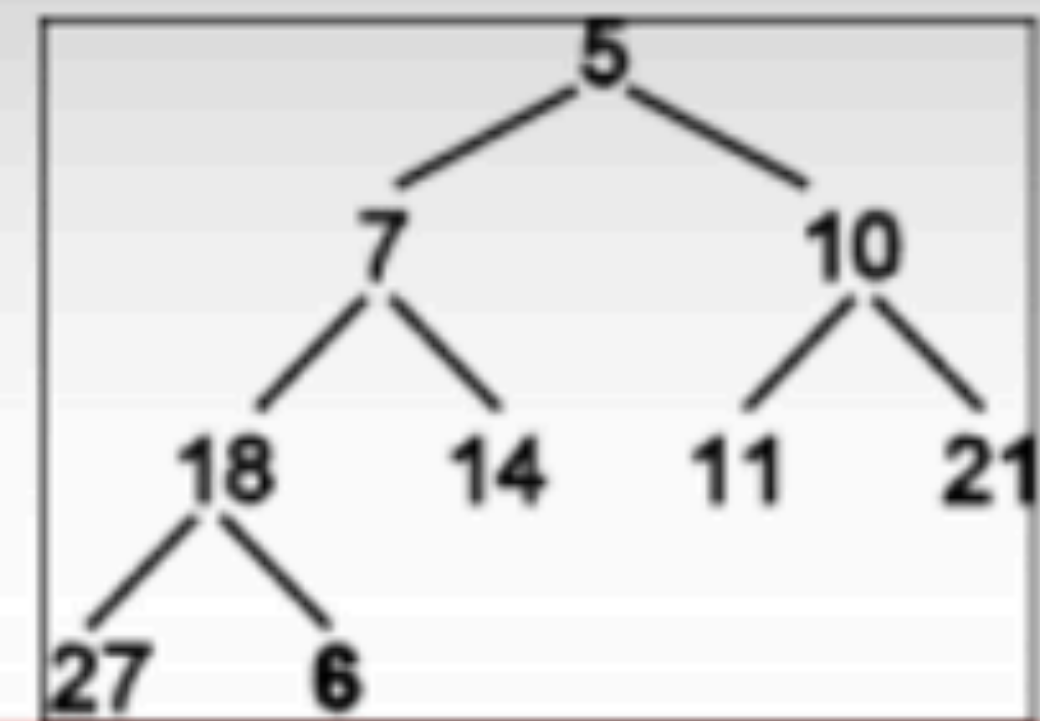




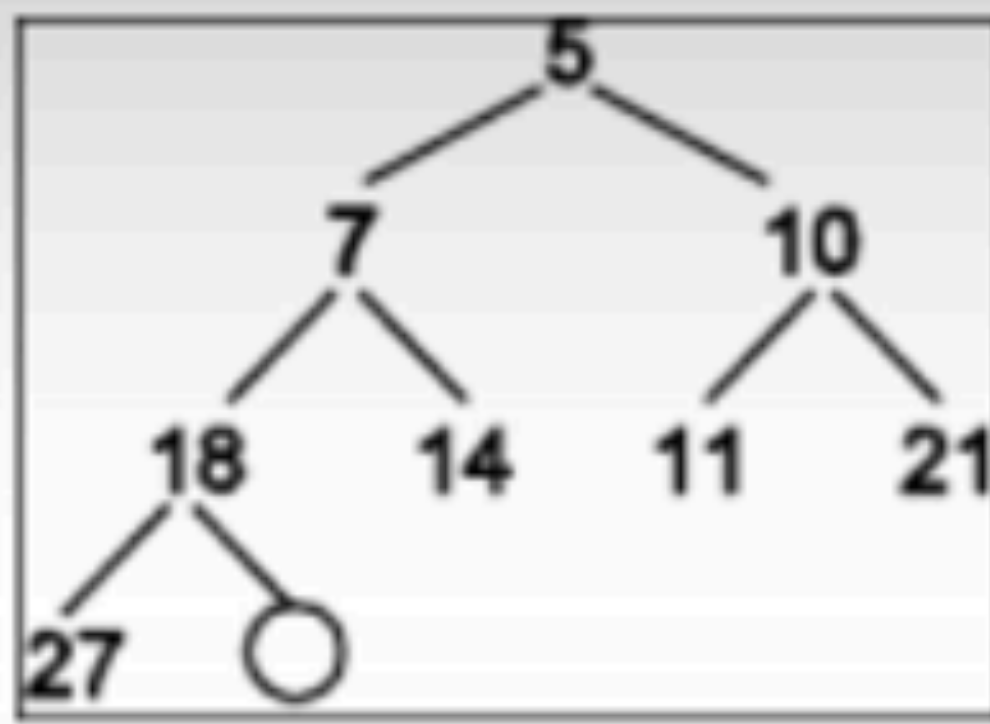
(a) A minheap prior to adding an element. The circle is where the new element will be put initially.



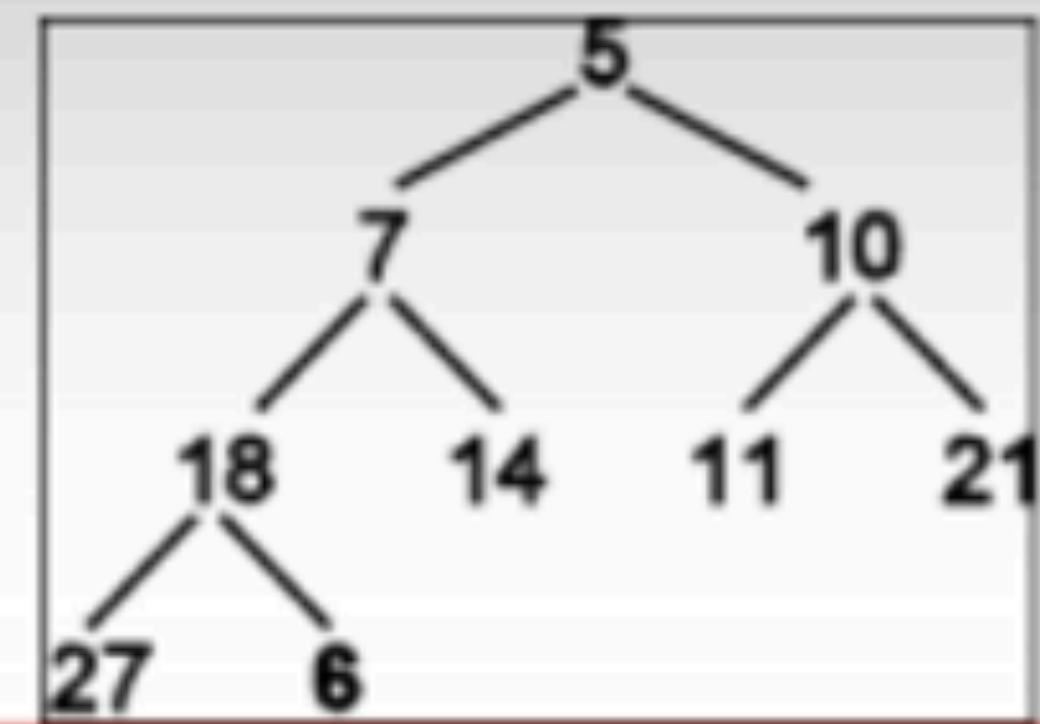
(a) A minheap prior to adding an element. The circle is where the new element will be put initially.



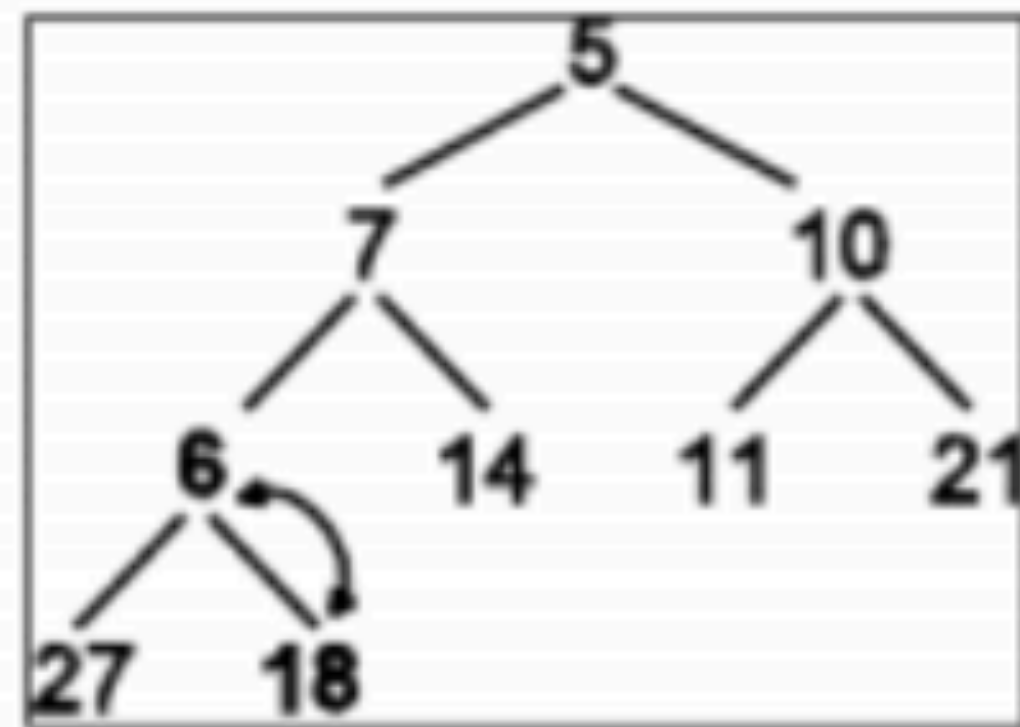
(b) Add the element, 6, as the new rightmost leaf. This maintains a complete binary tree, but may violate the minheap ordering property.



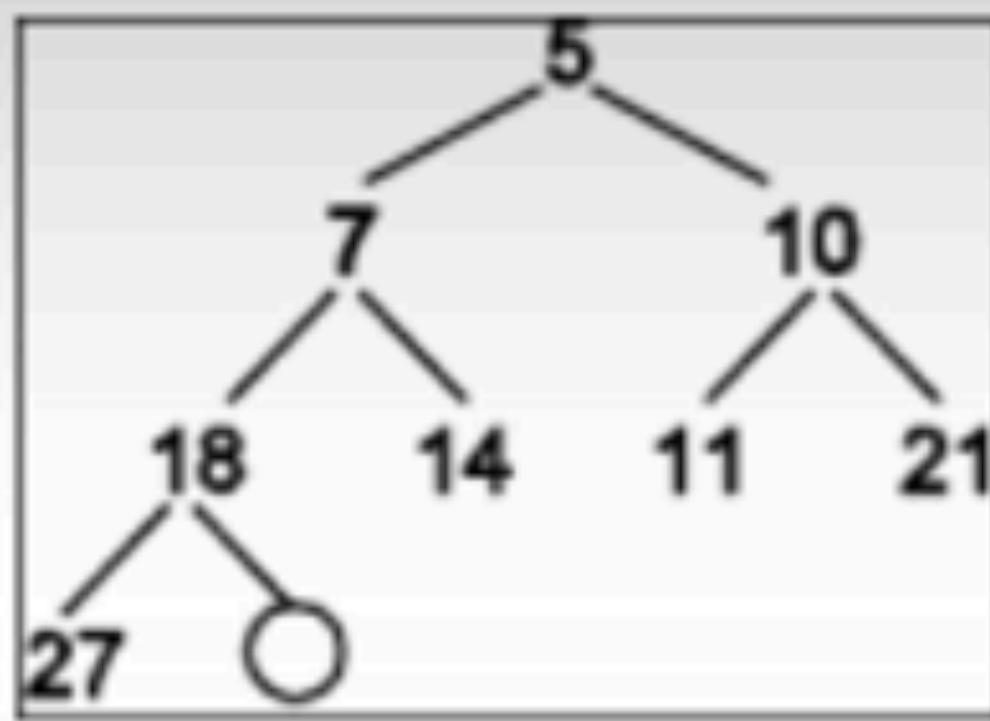
(a) A minheap prior to adding an element. The circle is where the new element will be put initially.



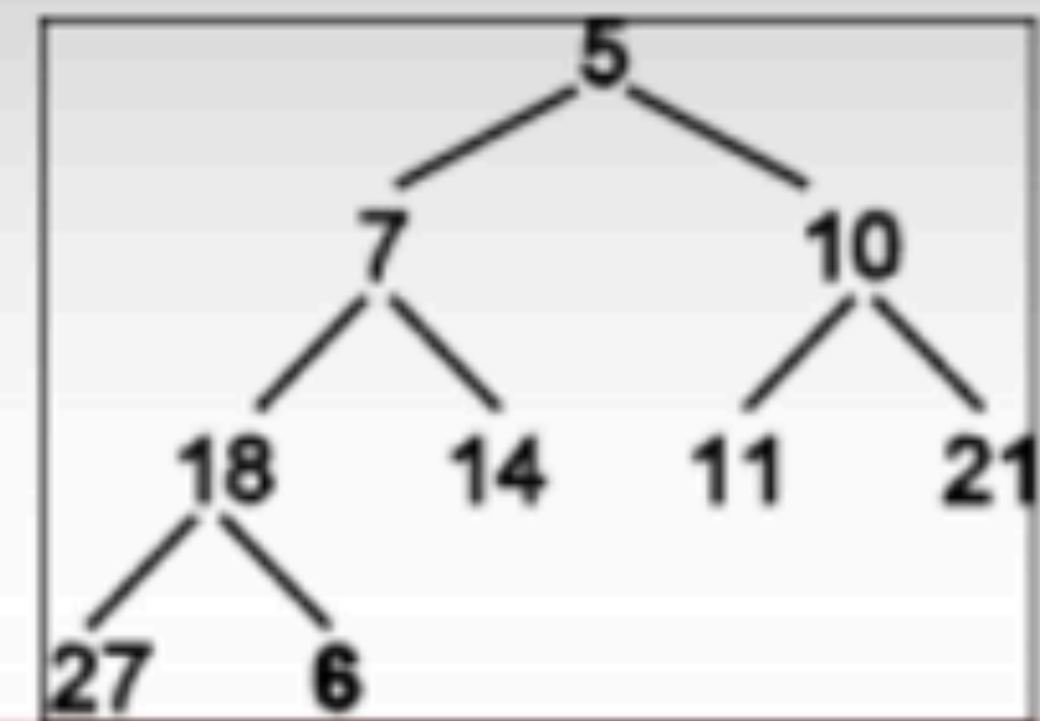
(b) Add the element, 6, as the new rightmost leaf. This maintains a complete binary tree, but may violate the minheap ordering property.



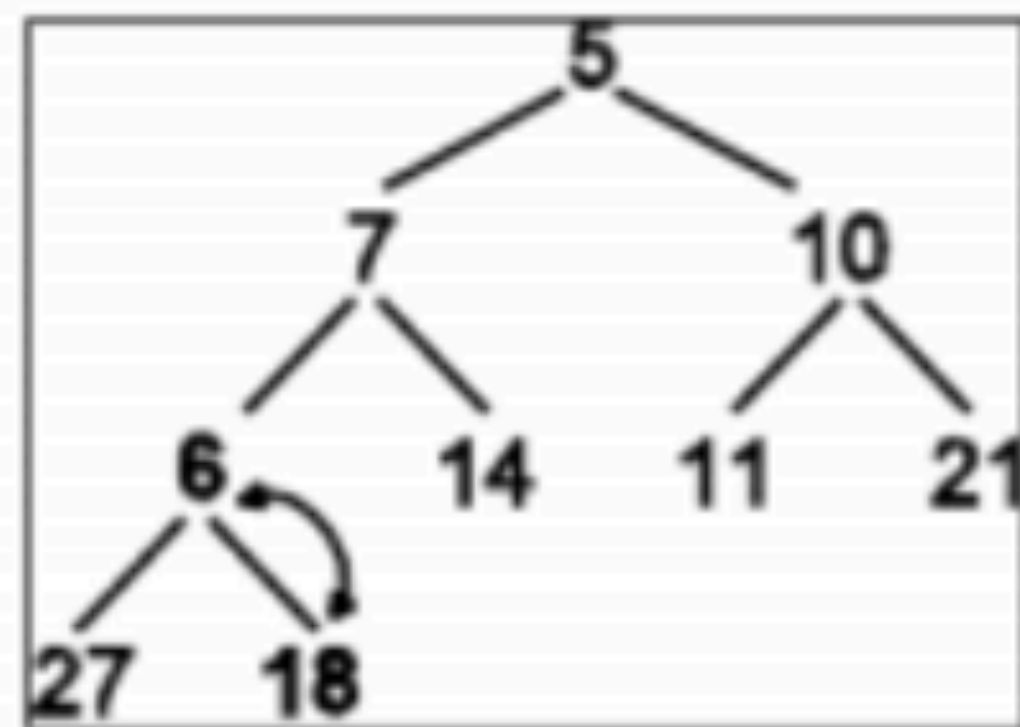
(c) "Bubble up" the new element. Starting with the new element, if the child is less than the parent, swap them. This moves the new element up the tree.



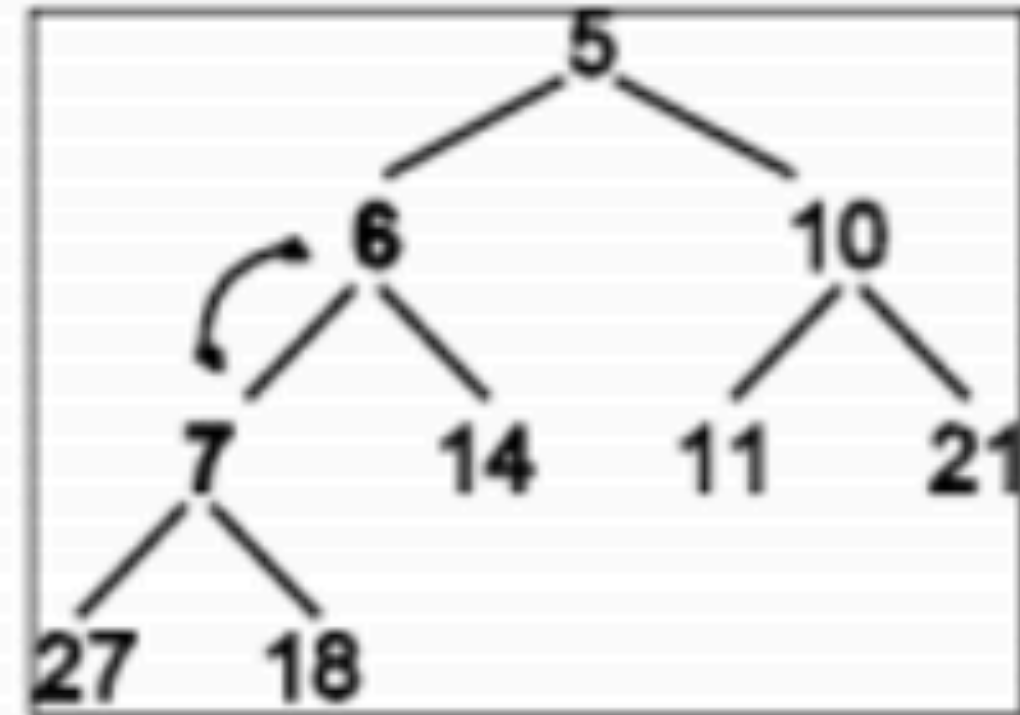
(a) A minheap prior to adding an element. The circle is where the new element will be put initially.



(b) Add the element, 6, as the new rightmost leaf. This maintains a complete binary tree, but may violate the minheap ordering property.

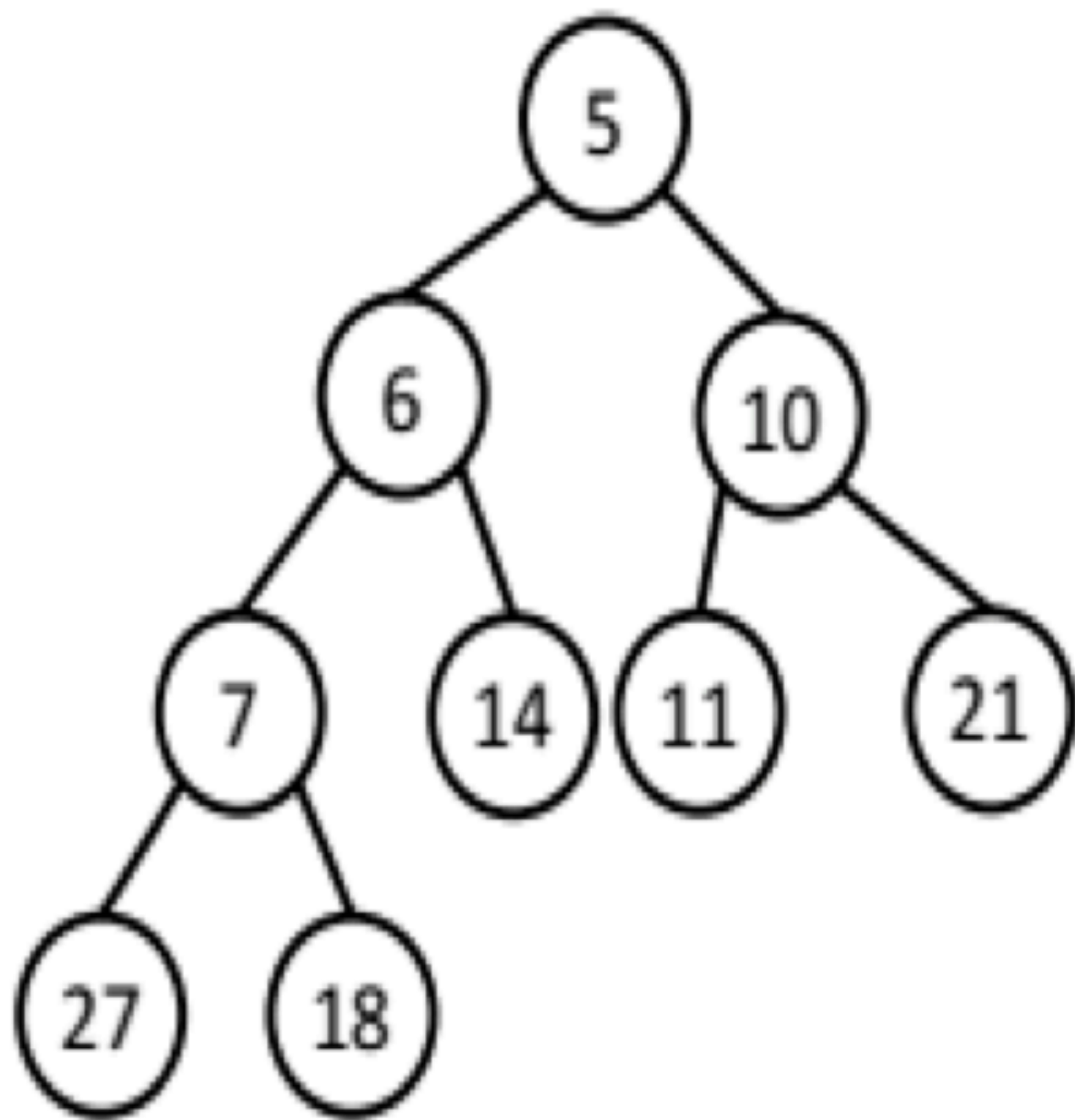


(c) “Bubble up” the new element. Starting with the new element, if the child is less than the parent, swap them. This moves the new element up the tree.



(d) Repeat the step described in (c) until the parent of the new element is less than or equal to the new element. The minheap invariants have been restored.

Insertion requires recursion



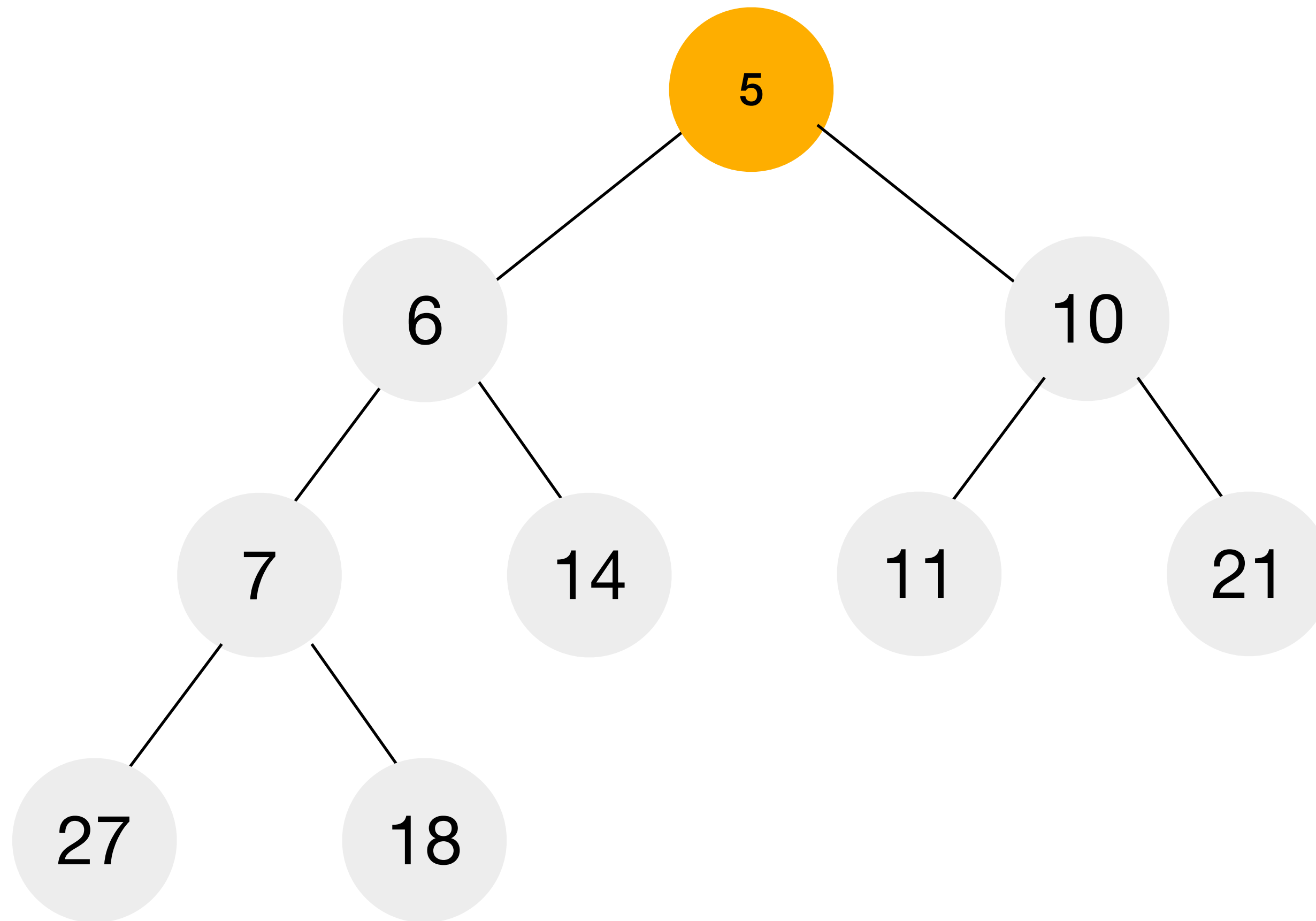
5	6	10	7	14	11	21	27	18	
0	1	2	3	4	5	6	7	8	

size

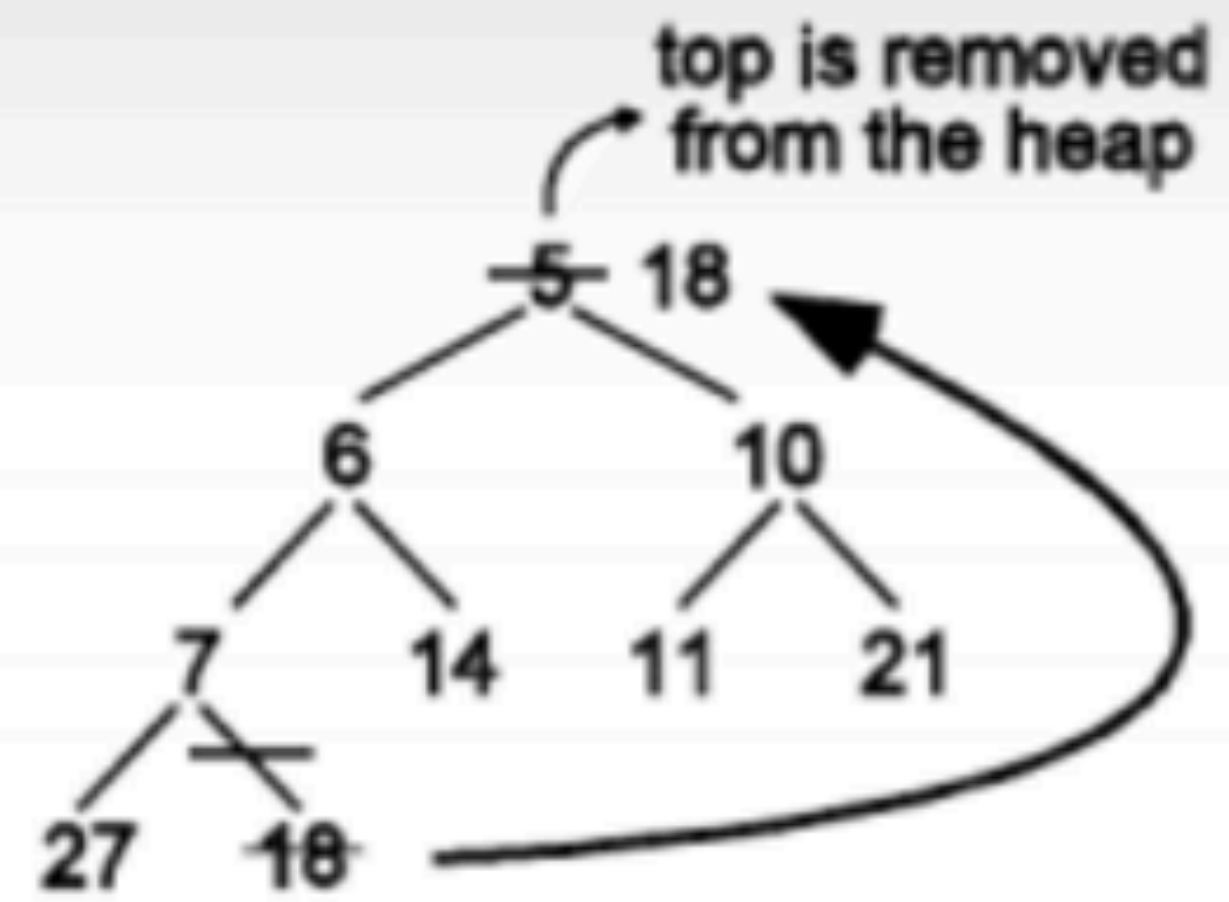
9

```
def BubbleUp(self, index):  
    if self.heaplist[index] > self.heaplist[(i-1)/2]:  
        # shouldn't move any more, done (return).  
    else  
        # Swap heaparray[index] with its parent  
        # self.Bubbleup(new_index)
```

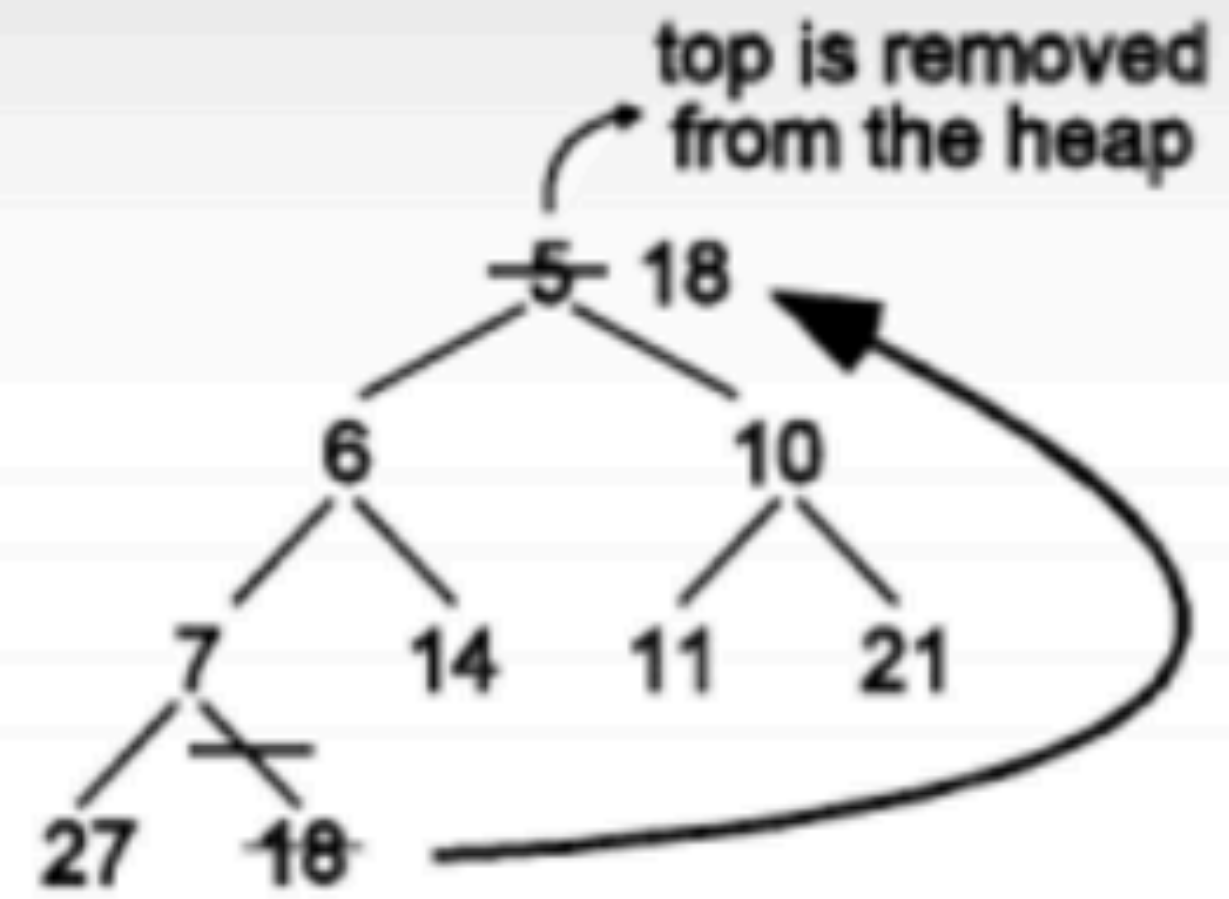
Deletion: insert the largest then bubble-down



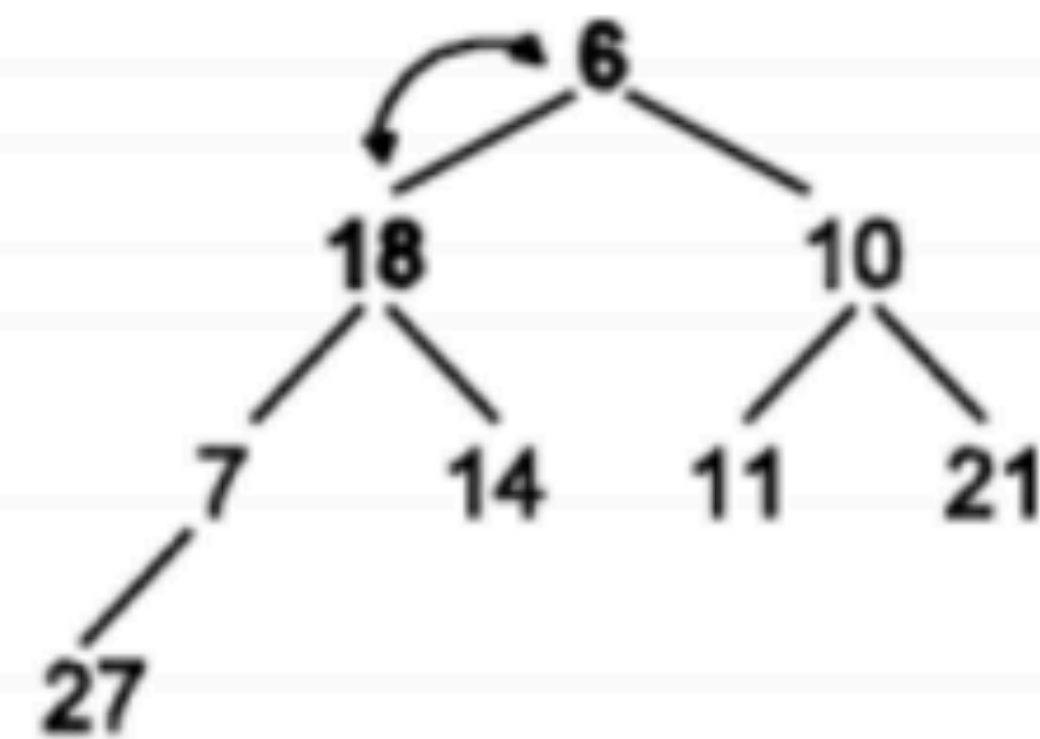
(a) Moving the rightmost leaf to the top of the heap to fill the gap created when the top element (5) was removed. This is a complete binary tree, but the minheap ordering property has been violated.



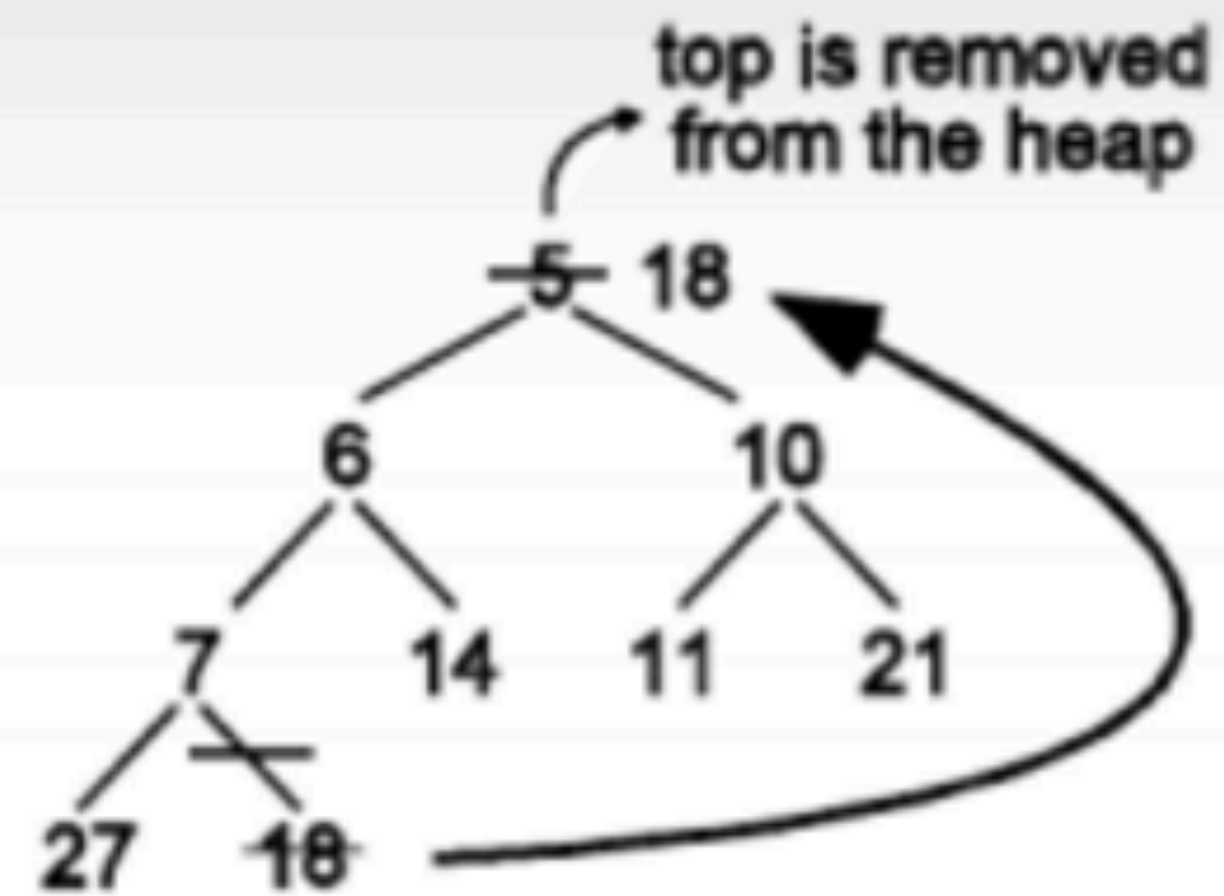
(a) Moving the rightmost leaf to the top of the heap to fill the gap created when the top element (5) was removed. This is a complete binary tree, but the minheap ordering property has been violated.



(b) “Trickle down” the element. Swapping top with the smaller of its two children leaves top’s right subtree a valid heap. The subtree rooted at 18 still needs fixing.



(a) Moving the rightmost leaf to the top of the heap to fill the gap created when the top element (5) was removed. This is a complete binary tree, but the minheap ordering property has been violated.



(b) "Trickle down" the element. Swapping top with the smaller of its two children leaves top's right subtree a valid heap. The subtree rooted at 18 still needs fixing.



(c) Last swap. The heap is fixed when 18 is less than or equal to both of its children. The minheap invariants have been restored

